

Differentiable Packing of Irregular 3D Objects with Adaptive Container Estimation

Palak Gupta^a, Shanmuganathan Raman^a

^a*Indian Institute of Technology Gandhinagar, India*

Abstract

Most existing approaches either fix the container in advance or optimize only a single container dimension through an outer search loop, leaving the remaining dimensions as a manual tuning problem. We present a differentiable packing framework that jointly optimizes all $6N$ object pose parameters and all three container side lengths inside a single gradient-based loop. The formulation combines six physics-inspired, differentiable loss terms computed directly on triangle meshes through axis-aligned bounding-box proxies. An adaptive squeezing mechanism periodically tightens the container whenever the overlap loss falls below a pair-count-scaled threshold, producing a large initial drop in container volume, followed by small refinements. All pairwise computations are written in tensor-broadcasting form, giving a 3.4 to 54 times speedup over a reference loop-based implementation. The pipeline is implemented in Python and PyTorch, with no physics engine, FFT library, or convex decomposition. On multiple object categories, the method produces containers that are 11 to 32 percent smaller than time-matched DBLF and simulated-annealing baselines at $N=100$, while running in under 4 minutes per instance on a single consumer GPU.

Keywords: 3D packing, irregular objects, differentiable optimisation, container estimation, adaptive squeezing, PyTorch

1. Introduction

The 3D packing problem asks to arrange a set of objects inside a container without overlap, typically as tightly as possible. For axis-aligned cuboids, the problem has been studied for decades [1, 2]. Real-world objects, however, are rarely cuboids: mechanical parts, kitchen utensils, 3D-printed assemblies, and consumer products are geometrically irregular and often concave. Discrete placement rules leave large voids on such inputs, and methods that replace irregular meshes with their axis-aligned bounding boxes as a permanent input proxy are bounded by the looseness of that replacement. Our method evaluates AABBs only as a per-iteration overlap surrogate computed from the transformed mesh vertices; the triangle mesh itself drives every pose update.

Email addresses: palak.gupta@iitgn.ac.in (Palak Gupta), shanmuga@iitgn.ac.in (Shanmuganathan Raman)

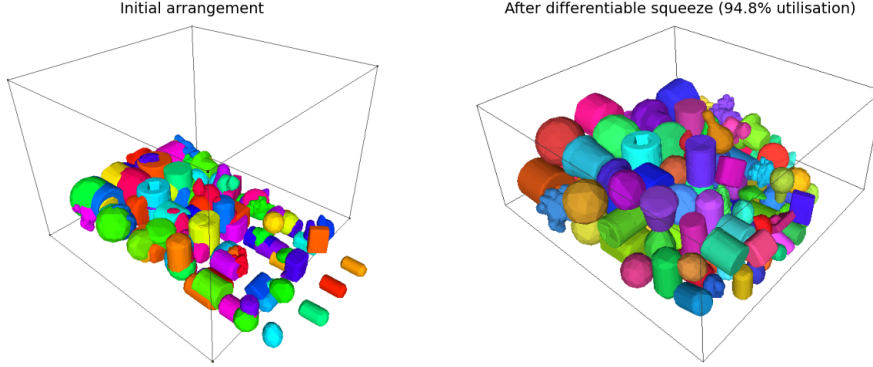


Figure 1: Our differentiable framework jointly optimizes object poses and all three container dimensions in a single gradient-based loop. **Left:** initial grid arrangement at $N=100$ kitchen objects inside the loose estimated container. **Right:** converged arrangement after pose optimization and adaptive container squeezing, achieving 94.8% container utilization with zero true mesh–mesh intersection.

Two variants dominate the literature. The *bin packing problem* (BPP) asks how many objects fit inside a container of given dimensions. The *open dimension problem* (ODP) asks to minimize one or more container dimensions while still accommodating every object. Existing approaches address these variants through constructive heuristics [3, 4], metaheuristic search [5], rigid-body simulation [6], continuous optimisation [7, 8], and reinforcement learning [10].

Irregular packing arises particularly in additive manufacturing, where build-plate utilisation directly affects machine time [16, 19]. Leao et al. [24] provide a comprehensive review of mathematical models for irregular packing problems. Hopper and Turton [20] survey 2D strip packing, which shares structural similarities with the 3D case.

The gap. Existing ODP methods share two structural limitations. **(i)** Container optimisation is one-dimensional: Zhuang et al. [6] optimise only the height; Ma et al. [7] run a binary search on a single dimension, invoking a continuous optimiser at every step. The other two dimensions are fixed in advance. **(ii)** The strongest recent results [6, 9] are built on C++ and CUDA pipelines combining NVIDIA PhysX with cuFFT, which are not easy to reproduce, extend, or integrate into a PyTorch or JAX workflow.

Our approach. We present a differentiable packing framework that optimizes the full $6N$ -dimensional pose vector jointly with all three container side lengths in a single continuous loop. The method operates in two modes sharing the same inner engine: a *fixed-container* mode for the BPP, and a *minimum container estimation* mode that progressively shrinks the container via an adaptive squeezing mechanism. Objects are represented directly as triangle meshes, with no voxelization, signed distance fields, or convex decomposition. Six physics-inspired differentiable loss terms drive the optimization, and every pairwise interaction is expressed as a broadcast tensor operation, so the forward pass contains no Python-level pair loops. Figure 2

summarises the pipeline.

Contributions.

1. A differentiable formulation of 3D irregular packing with six physics-motivated loss terms (overlap, boundary, support-aware gravity, contact, cohesion, centripetal), each with a meaningful gradient through the rotation and translation parameters.
2. An adaptive container squeezing mechanism with a pair-count-scaled overlap threshold that makes the same procedure work reliably from $N=10$ to $N>100$ without per- N tuning.
3. A fully vectorized PyTorch implementation that replaces the $\binom{N}{2}$ Python loop with tensor broadcasting, yielding a 3.4 to 54 times speedup and allowing a 100-object instance to finish in about three minutes on a single consumer GPU.
4. A systematic evaluation on multiple object categories against three time-matched baselines. At $N=100$, our method produces containers 11 to 32 percent smaller than the best of DBLF, simulated annealing, and BLF+SA.

Reproducibility. The entire framework depends only on PyTorch, NumPy, SciPy, and `trimesh`. No compiled extensions, custom CUDA kernels, physics engine bindings, or FFT libraries are used. All hyperparameters are reported in Section 4, and all results are generated with five fixed random seeds.

2. Related Work

Constructive heuristics. Wang et al. [3] introduced Deepest-Bottom-Left-Fill (DBLF), a sequential greedy placement rule. Wang and Hauser [4] extended DBLF with a heightmap-minimizing step. The same authors address robotic packing with nondeterministic item arrival [22], a related but distinct online setting. Crainic et al. [11] generalized such rules through an extreme-point framework. DBLF-family methods are fast and yield zero-overlap arrangements by construction, but their greedy nature makes them dependent on placement order, and they cannot revisit earlier decisions. Garey et al. [21] established the NP-hardness of bin packing, which motivates the use of heuristic and continuous optimisation approaches.

Metaheuristic search. Simulated annealing [5] and genetic algorithms [12] have been applied by encoding a candidate solution as the object sequence passed to a DBLF-like inner placer. These methods escape strict greediness but remain bottlenecked by the inner discrete placer and by the curse of dimensionality as N grows.

Physics-based simulation. Zhuang et al. [6] combine DBLF initialization with FFT-based collision detection [9] and rigid-body dynamics in NVIDIA PhysX. Horizontal shaking drives objects into compact arrangements. The pipeline is built on C++, CUDA, PhysX, and cuFFT, and optimizes only the container height.

Continuous optimisation. Ma et al. [7] use continuous optimization of positions and orientations followed by combinatorial object-swapping, running a binary search over the height dimension and reporting 28 to 58 minutes for 60 to 130 objects. Stoyan et al. [18] develop quasi-phi-functions for continuous rotations, on which Romanova et al. [8] build for concave polyhedra but scaling to only about 10 objects.

Spectral and learning-based methods. Cui et al. [9] voxelise objects and use FFT-based spectral correlation, restricting rotations to a discrete set and assuming a fixed container. Learning-based approaches [10, 14] train RL agents for online BPP with a fixed container; Hu et al. [15] apply DRL to cuboid packing. All three settings assume a fixed container and are not directly comparable to our offline container estimation problem.

Geometry processing and fabrication-aware packing. Attene [26] introduces a 3D packing algorithm that splits objects into parts for tight box-packing with minimum container volume, addressing the same NP-hard problem class as our work from a fabrication-oriented perspective. Cao et al. [28] address constrained 3D stacking for additive manufacturing, proposing a voxel-based layout method for DLP printing that highlights the importance of physical constraints in packing. Krs et al. [27] present a procedural iterative constrained optimiser for geometric modelling that combines gradient-based optimisation with geometric constraints, closely aligned with our differentiable formulation.

Positioning. Our method differs in three ways: all three container dimensions are jointly optimized; optimization runs directly on triangle meshes via per-epoch AABBs, without voxelization or FFT grid; and the pipeline runs in pure Python and PyTorch.

3. Method

3.1. Overview

Given N triangle-mesh objects, we seek a position $\mathbf{p}_i \in \mathbb{R}^3$ and Euler-angle rotation $\boldsymbol{\theta}_i \in \mathbb{R}^3$ for every object such that (i) no two objects overlap, (ii) all objects lie inside a rectangular container whose volume is minimised, and (iii) each object rests on the ground plane or another object. The framework (Figure 2) has four blocks: an input block that loads meshes and estimates an initial container; a loss block that computes six differentiable loss terms from cached AABBs; an optimization block that steps two Adam optimizers over the $6N$ pose parameters; and a squeeze block that monitors the overlap loss and updates the container when triggered. Algorithm 1 gives the pseudo-code.

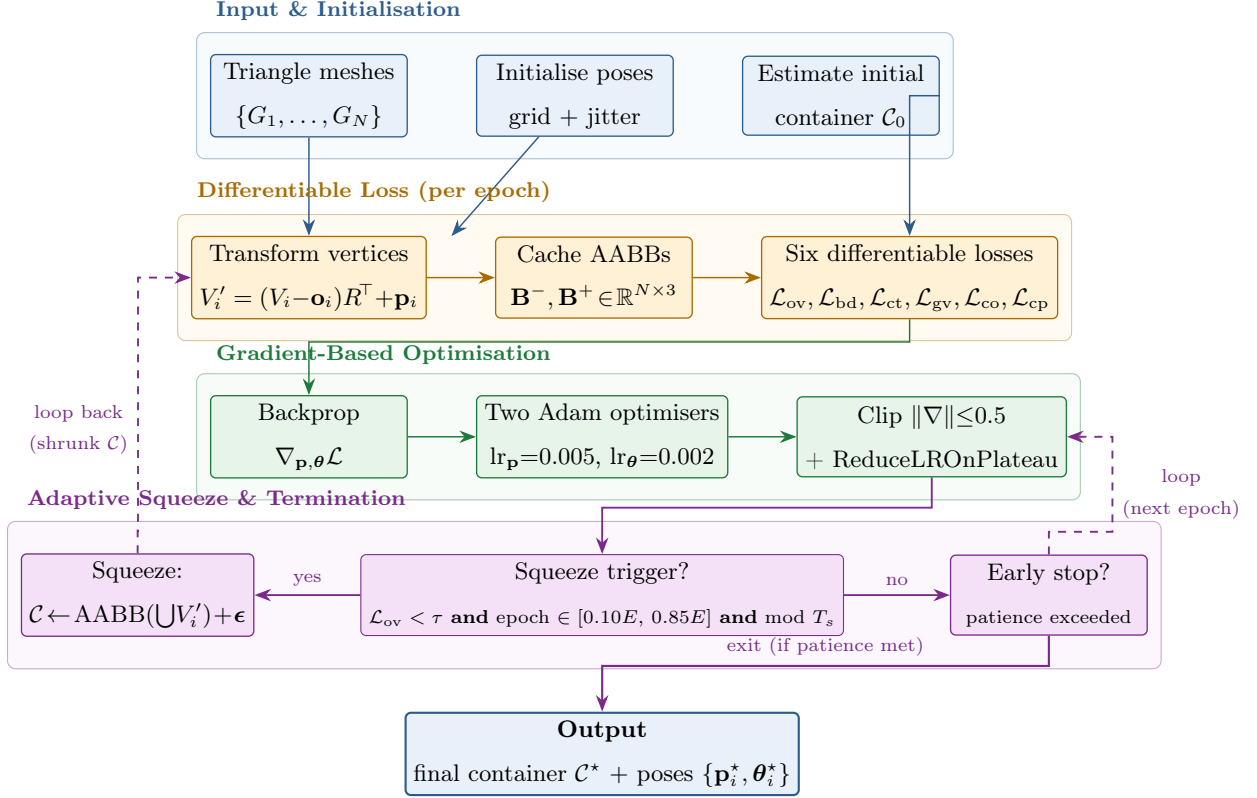


Figure 2: Pipeline architecture. Meshes and the initial container enter the loss block (amber), which caches per-object AABBs from the transformed vertices and evaluates six differentiable loss terms. Gradients flow into the optimization block (green). The squeeze block (purple) tightens the container when the overlap loss is below the pair-count-scaled threshold τ . Early stopping exits the loop when no improvement is observed for the patience window.

3.2. Object Representation and Transformation

Each mesh $G_i = (V_i, F_i)$ is re-centered at its object-local bounding-box centroid $\mathbf{o}_i = \frac{1}{2}(\min V_i + \max V_i)$. For Euler angles $\boldsymbol{\theta}_i = (\theta_x, \theta_y, \theta_z)$ in ZYX convention, the rotation matrix is

$$R(\boldsymbol{\theta}_i) = R_z(\theta_z) R_y(\theta_y) R_x(\theta_x) \in SO(3), \quad (1)$$

and the transformed vertices are

$$V'_i = (V_i - \mathbf{o}_i) R(\boldsymbol{\theta}_i)^\top + \mathbf{p}_i. \quad (2)$$

The instantaneous AABB is $\mathbf{b}_i^- = \min V'_i$, $\mathbf{b}_i^+ = \max V'_i$, recomputed every iteration from the transformed vertices. Rotating a precomputed object-local AABB would yield a strict over-approximation of the rotated object’s AABB and forfeit the geometric information that makes tight irregular packing possible.

Each object contributes 6 learnable parameters; the full search space has dimension $6N$. Objects are sorted by volume in decreasing order and placed on a coarse grid of side $G = \lceil N^{1/3} \rceil - 1$ with Gaussian jitter ($\sigma=0.1$) in the x and z axes; y is set to the ground plane. Sorting by volume empirically reduces the

Algorithm 1 Differentiable packing with adaptive squeezing.

Require: meshes $\{G_1, \dots, G_N\}$, initial container $\mathcal{C}_0$, max epochs $E_{\max}$

Ensure: final container $\mathcal{C}^*$, poses $\{(\mathbf{p}_i^*, \boldsymbol{\theta}_i^*)\}$

```

1: initialise  $\{\mathbf{p}_i\}$  on a compact grid with Gaussian jitter; set  $\{\boldsymbol{\theta}_i\} \leftarrow \mathbf{0}$ 
2: set squeeze threshold  $\tau \leftarrow \max(5.0, 0.005 \cdot N(N-1)/2)$ 
3: set squeeze window  $[E_{\text{lo}}, E_{\text{hi}}] \leftarrow [\lfloor 0.10E_{\max} \rfloor, \lfloor 0.85E_{\max} \rfloor]$ 
4: set squeeze period  $T_s \leftarrow \max(40, \lfloor E_{\max}/20 \rfloor)$ 
5: for  $e = 1, \dots, E_{\max}$  do
6:   transform vertices:  $V'_i \leftarrow (V_i - \mathbf{o}_i)R(\boldsymbol{\theta}_i)^\top + \mathbf{p}_i$ 
7:   cache AABBs; evaluate  $\mathcal{L}_{\text{total}}$  via vectorised broadcasts
8:   backpropagate; clip  $\|\nabla\|_2 \leq 0.5$ ; step both Adam optimisers
9:   if  $\mathcal{L}_{\text{ov}} < \tau$  and  $e \in [E_{\text{lo}}, E_{\text{hi}}]$  and  $e \bmod T_s = 0$  then
10:     $\mathcal{C} \leftarrow \text{AABB}(\bigcup_i V'_i) + \epsilon$  {squeeze}
11:   end if
12:   if early-stop patience exceeded then
13:    break
14:   end if
15: end for
16: return  $\mathcal{C}^*, \{(\mathbf{p}_i^*, \boldsymbol{\theta}_i^*)\}$ 

```

number of epochs spent unwedging a large object from between smaller neighbors. Rotations are initialized to $\mathbf{0}$.

3.3. Physics-Inspired Loss Functions

The total loss is a weighted sum of six terms:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ov}} + \mathcal{L}_{\text{bd}} + \frac{1}{\sqrt{N}} \mathcal{L}_{\text{ct}} + \mathcal{L}_{\text{gv}} + \mathcal{L}_{\text{co}} + \mathcal{L}_{\text{cp}}. \quad (3)$$

The $1/\sqrt{N}$ prefactor on the contact term prevents a quadratic-in- N attractive force from overwhelming the overlap term at large N .

Overlap Loss. Per-axis overlap between two AABBs is $o_{ij}^k = \max(0, \min(b_{i,k}^+, b_{j,k}^+) - \max(b_{i,k}^-, b_{j,k}^-))$ for $k \in \{x, y, z\}$, and

$$\mathcal{L}_{\text{ov}} = w_{\text{ov}} \sum_{i < j} \left(\prod_k o_{ij}^k \right) \mathbf{1}[\forall k : o_{ij}^k > \epsilon], \quad (4)$$

with $w_{\text{ov}} = 80 \ln(N+1)$ and $\epsilon = 10^{-6}$. The log scaling keeps the term dominant during the early phase of optimization, independent of N .

Boundary Loss. Objects must stay inside the container $[\mathbf{c}^-, \mathbf{c}^+]$, with an amplified ground term:

$$\mathcal{L}_{\text{bd}} = w_{\text{bd}} \sum_i \left[100 \sum_k (\max(0, c_k^- - b_{i,k}^-)^2 + \max(0, b_{i,k}^+ - c_k^+)^2) + 1000 \max(0, -b_{i,y}^-)^2 \right]. \quad (5)$$

Support-Aware Gravity Loss. A naive penalty on $b_{i,y}^-$ would prevent stacking. Instead, the effective supporting height is computed from objects beneath whose horizontal footprints overlap by at least η :

$$\mathcal{S}_i = \{ j : o_{ij}^x > \eta, o_{ij}^z > \eta, b_{j,y}^+ \leq b_{i,y}^- \}, \quad g_i = \max(0, \max_{j \in \mathcal{S}_i} b_{j,y}^+). \quad (6)$$

With floating gap $\gamma_i = \max(0, b_{i,y}^- - g_i)$,

$$\mathcal{L}_{\text{gv}} = \sum_i (20\gamma_i^2 + 5\gamma_i) \mathbf{1}[\gamma_i > 10^{-3}] + 2000 \sum_i \max(0, -b_{i,y}^-)^2. \quad (7)$$

This produces emergent multi-layer stacking without any explicit heuristic.

Contact, Cohesion, Centripetal. With per-axis gap d_{ij}^k ,

$$\mathcal{L}_{\text{ct}} = w_{\text{ct}} \sum_{i < j} \sum_k (d_{ij}^k)^2 \mathbf{1}[\|d_{ij}\| < d_{\text{max}}], \quad (8)$$

where $d_{\text{max}} \approx 1$ prevents the attractive force from collapsing distant objects. The cohesion term is a volume-weighted center-of-mass attractor with $\bar{\mathbf{p}} = \sum_i v_i \mathbf{p}_i / \sum_i v_i$, and centripetal pulls the cluster toward the container axis in the xz plane:

$$\mathcal{L}_{\text{co}} = w_{\text{co}} \sum_i \|\mathbf{p}_i - \bar{\mathbf{p}}\|_2, \quad \mathcal{L}_{\text{cp}} = w_{\text{cp}} \sum_i \|\mathbf{p}_i^{xz} - \mathbf{c}^{xz}\|_2. \quad (9)$$

Weights are $w_{\text{ov}}=80 \ln(N+1)$, $w_{\text{bd}}=5.0$, $w_{\text{ct}}=2.0$, $w_{\text{co}}=0.5$, $w_{\text{cp}}=0.3$.

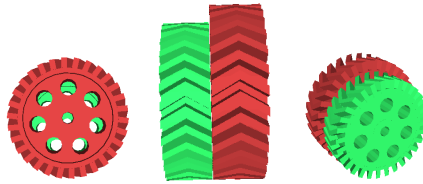


Figure 3: Two gear objects are arranged in contact with each other (showing front, side, and back-top view), indicating tight packing with no overlaps.

3.4. Vectorized Pairwise Computation and Adaptive Squeezing

Each loss term with an $i < j$ sum naively requires a double Python loop over $\binom{N}{2}$ pairs. We eliminate these loops by caching the AABBs as tensors $\mathbf{B}^-, \mathbf{B}^+ \in \mathbb{R}^{N \times 3}$ and expressing every pairwise quantity through broadcasting:

$$\mathbf{O}_{ij,k} = \max(0, \min(B_{i,k}^+, B_{j,k}^+) - \max(B_{i,k}^-, B_{j,k}^-)), \quad (10)$$

evaluated for all pairs in a single broadcast, producing an $N \times N \times 3$ tensor in one CUDA kernel launch. An upper-triangular mask selects unique pairs. The same pattern is applied to the contact gaps and the xz footprints used by the gravity term. The speedup grows from $3.4\times$ at $N=10$ to $54\times$ at $N=100$.

Adaptive squeeze. Once overlaps are substantially resolved, the container is snapped to the tight AABB of all transformed vertices plus a margin:

$$\mathcal{C}_{\text{new}} = \text{AABB}(\bigcup_i V'_i) + \epsilon, \quad \epsilon = \max(0.08, 0.15/\sqrt[3]{N}). \quad (11)$$

The $N^{-1/3}$ scaling follows from a dimensional argument: the characteristic single-object length scales as the cluster extent divided by $N^{1/3}$. A squeeze event fires only when the epoch is in $[0.10E_{\text{max}}, 0.85E_{\text{max}}]$, is a multiple of $T_s = \max(40, \lfloor E_{\text{max}}/20 \rfloor)$, and

$$\mathcal{L}_{\text{ov}} < \tau, \quad \tau = \max(5.0, 0.005 \cdot \frac{N(N-1)}{2}). \quad (12)$$

The pair-count scaling of τ is key to operating reliably across N : a constant $\tau=5$ is strict at $N=10$ (45 pairs) and essentially always violated at $N=100$ (4,950 pairs). The scaled form converts τ into a bound on mean overlap per pair. Five to ten squeeze events typically occur during a single run. Figure 4 illustrates the dynamics on kitchen at $N=60$. The loss drops three orders of magnitude in the first 300 epochs as gross overlaps are resolved. The first squeeze fires around epoch 450 once $\mathcal{L}_{\text{ov}} < \tau$, shrinking the container from roughly 35.5 to 19.7 in a single step. Subsequent squeezes make much smaller adjustments, and the container volume rises slightly between events as the optimizer trades off a tiny amount of tightness for the boundary and gravity terms it has to re-satisfy after each shrink. The final converged volume of around 22.8 reflects this equilibrium rather than continued monotone shrinking.

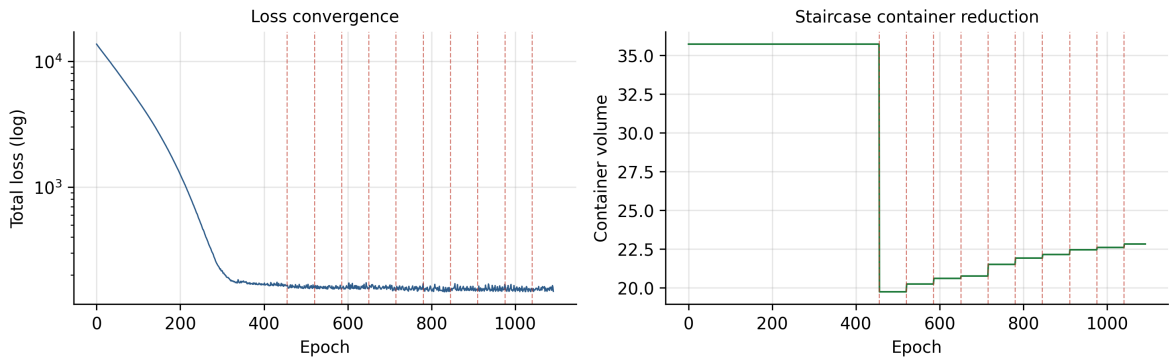


Figure 4: Convergence dynamics on kitchen at $N=60$. Left: total loss (log scale). Right: container volume. Red dashed lines mark squeeze events. The first squeeze (epoch ~ 450) delivers the large drop; later events stabilize the container at its equilibrium volume.

3.5. Optimiser Configuration

Two Adam [13] optimizers are used: lr= 0.005 for translations, lr= 0.002 for rotations (lower because a one-radian rotation moves far vertices by up to one unit while a unit translation moves every vertex by exactly one unit). Both use a `ReduceLROnPlateau` scheduler (patience 100, factor 0.7). Gradients are clipped to L_2 norm 0.5. Training runs for $E_{\max} = \min(5000, 1000 + 5N)$ epochs with early stopping after $\min(500, 200 + N)$ epochs of no improvement. The initial container has volume $\sum_i v_i / 0.35$ with aspect ratio (1.1, 0.85, 1.1). In fixed-container mode, the squeeze trigger is disabled, and the initial container is used throughout training; all other components are identical.

4. Experimental Setup

4.1. Implementation and Datasets

The framework uses Python 3 and PyTorch [23] for autodiff and GPU tensor operations, with `trimesh` [17] for mesh loading and `scipy.spatial.ConvexHull` for post-hoc hull volumes. Experiments run on a workstation with a consumer-grade GPU; each run uses a single GPU in `float32`. All runs are seeded.

We evaluate on multiple object categories:

- **Kitchen:** cups, bowls, plates, mugs, utensils; hollowness $h \approx 0.54$.
- **Gear:** gear models with concave tooth profiles; $h \approx 0.40$.
- **Block:** near-solid number-shaped blocks; $h \approx 0.88$.
- **Mixed:** a union of the above; $h \approx 0.55$.

Hollowness $h = V_{\text{mesh}}/V_{\text{AABB}}$ is the ratio of mesh volume to bounding-box volume and matters when interpreting density: a thin-walled object cannot match a solid block’s density even in principle. Each object is rescaled by its longest bounding-box edge to unit characteristic size, then recentered. Datasets are sampled with replacement and random scale in $[0.4, 0.8]$ to produce sets of size $N \in \{10, 30, 60, 100\}$. We have also evaluated various other shapes, implying that our framework can be implemented on any 3D object.

4.2. Evaluation Metrics

Container volume $V = s_x s_y s_z$ is the headline metric (lower better). **Packing density** is $V_{\text{mesh}}/V_{\text{cont}}$, bounded above by h . **Packing efficiency** is the ratio of the sum of per-object convex-hull volumes to the global convex-hull volume, independent of container choice. **Container utilisation** is the ratio of the packed-cluster AABB to the container; values near 100% indicate a tight squeeze, values above 100% indicate overflow. **AABB overlap rate** is the fraction of pairs with intersection above a 2%-of-diagonal tolerance;

this is an upper bound on true mesh overlap. Every reported result was inspected in MeshLab and has zero true mesh intersection. **Hull density** is the ratio of the sum of per-object AABB volumes to the convex-hull volume of the cluster, measuring how tightly the arrangement approximates its own bounding hull. **Floating count** is the number of objects whose lowest vertex sits more than 0.05 unit-lengths above any supporting surface beneath them; it measures physical plausibility of the packing rather than its tightness.

5. Results I: Minimum-Container Packing (Squeeze Mode)

We report our method in adaptive-squeeze mode on four categories at four object counts. Every entry in Table 1 is a mean over five seeds.

Container utilisation is consistent. Container utilisation stays in the 81 to 95% range across every dataset and every N . This is the signature of a tight squeeze: the container is pulled onto the converged cluster, rather than being left loose or forced to overflow. The same pair-count-scaled trigger handles $N=10$ and $N=100$ without per-instance tuning.

Block stands apart for geometric reasons. Block reaches 64 to 71% packing efficiency at $N \geq 30$; kitchen, gear, and mixed peak at 42 to 55%. This reflects geometry, not the solver: block meshes are close to their AABBs ($h \approx 0.90$), so packing the AABB tightly also packs the mesh tightly. The solver-quality signal is two condition-independent metrics: hull density stays within 28 to 66% and container utilization within 81 to 95% across all categories.

Effect of N within each dataset. Packing density behaves non-monotonically with N . On kitchen it decreases from 55% at $N=10$ to 42% at $N=60$ before rising back to 47% at $N=100$. The dip at intermediate N reflects a geometric transition: at $N=10$ the cluster is essentially a single layer and the squeeze is bounded by the largest object’s footprint; at $N=60$ the cluster is tall enough that vertical voids open between stacking layers; by $N=100$ enough small objects exist to fill those voids, and density recovers. Block and gear follow the same pattern with different inflection points dictated by their hollowness factors. This non-monotonic behavior is a property of the geometry, not the solver: container utilization stays in the 81 to 95% range at every N , confirming the squeeze mechanism converges equally tightly regardless of which phase the cluster is in.

Overlap and runtime. AABB overlap rate stays at or below 0.4% on every cell, and 0.0% on blocks at $N \leq 30$. Every arrangement has zero true mesh intersection (Figure 5). Residual AABB overlaps are concave objects whose bounding boxes touch without the meshes intersecting. Runtime scales roughly linearly with N : from 24–37s at $N=10$ to 218–223s at $N=100$.

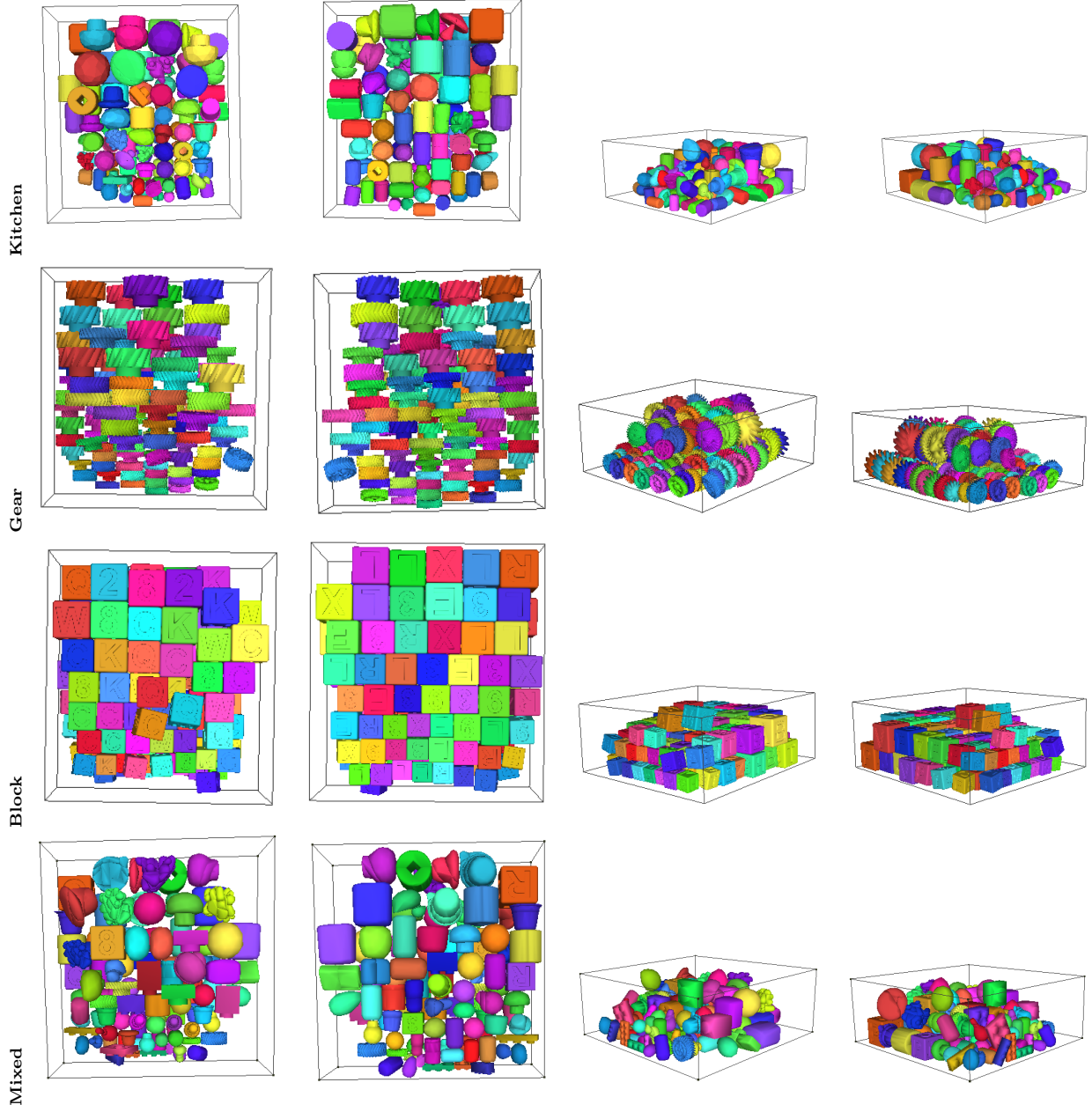


Figure 5: Squeeze-mode results at $N=100$. Four views (top, bottom, right, left) per dataset row. All configurations have zero true mesh-mesh overlap.

Table 1: Minimum-container (squeeze) packing results. All configurations have zero true mesh-mesh intersections, verified in MeshLab.

Dataset	n	Container	Pack Eff.%	Hull Dens.%	Cont. Util.%	Overall%	h	AABB Ov.	Mesh Ov.	Time
Kitchen	10	$1.604 \times 0.876 \times 1.961$	55.0	51.9	82.3	32.4	0.56	2.2	0	36.2s
	30	$2.761 \times 1.498 \times 2.602$	45.7	42.8	89.1	24.0	0.54	0.2	0	69.7s
	60	$3.739 \times 1.725 \times 3.860$	42.3	38.7	91.7	26.3	0.53	0.1	0	106.7s
	100	$4.126 \times 1.681 \times 4.583$	47.3	43.6	94.8	27.4	0.54	0.1	0	218.9s
Gear	10	$1.522 \times 0.866 \times 1.511$	63.1	38.5	81.3	26.9	0.43	0	0	24.3s
	30	$2.784 \times 1.585 \times 2.424$	47.3	28.0	88.7	14.9	0.41	0.2	0	58.0s
	60	$3.287 \times 1.629 \times 3.376$	48.7	28.5	90.6	15.9	0.40	0.3	0	132.8s
	100	$3.675 \times 1.583 \times 3.877$	51.2	29.8	89.4	20.2	0.40	0.1	0	220.8s
Block	10	$1.935 \times 1.522 \times 2.609$	64.5	60.6	88.1	38.6	0.88	0	0	32.9s
	30	$3.039 \times 1.616 \times 3.349$	70.9	66.4	90.7	51.5	0.87	0	0	64.8s
	60	$4.249 \times 1.859 \times 4.478$	67.6	63.3	91.0	42.3	0.88	0.2	0	133.9s
	100	$4.695 \times 2.012 \times 5.174$	67.7	63.3	94.6	45.7	0.86	0.1	0	223.1s
Mixed	10	$2.028 \times 1.387 \times 1.655$	55.0	47.6	85.8	26.3	0.63	0	0	35.3s
	30	$2.883 \times 1.484 \times 2.658$	50.2	42.3	89.3	27.3	0.51	0.2	0	70.4s
	60	$3.764 \times 1.589 \times 3.837$	47.0	40.2	92.2	24.5	0.52	0.3	0	132.7s
	100	$4.369 \times 1.753 \times 4.472$	45.3	39.8	92.5	25.3	0.54	0.2	0	221.0s

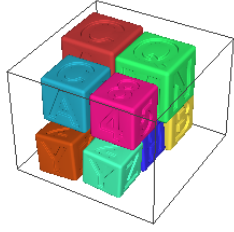
6. Results II: Fixed-Container Packing (No-Squeeze Mode)

In fixed-container mode the squeezing mechanism is disabled and container dimensions are specified as input. We evaluate on three categories at four N values. Container dimensions are chosen to provide a feasible but challenging packing problem. To reflect real-world variability, objects within each category are assigned random scale factors in $[0.4, 0.8]$, in contrast to prior work [6, 7] where instances share identical dimensions. Table 2 reports all metrics; Figure 6 visualises the arrangements.

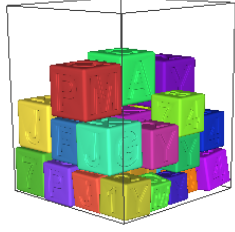
Blocks achieve the highest packing efficiency (74–80% at $n \leq 30$) with zero AABB overlaps, owing to their near-solid geometry; the AABB proxy closely approximates the actual mesh. **Gears exhibit AABB overlaps that are not mesh overlaps:** at $n=30$ and $n=60$, 16 AABB-overlapping pairs are reported despite zero actual mesh intersection, because interlocking teeth create overlapping bounding boxes even when the meshes maintain clearance. **Emergent multi-layer stacking** appears across all three datasets without explicit heuristics, driven by the interaction of support-aware gravity, contact attraction, and cohesion. **Container utilization above 100%** (gear $n=60$: 107.9%, kitchen $n=60$: 107.9%, kitchen $n=100$: 104.8%) indicates the packed AABB extends slightly beyond the container walls at contact points; visual inspection confirms all objects remain tangent to or inside the container.

Table 2: Fixed-container (no-squeeze) packing results. “AABB Ov.” reports axis-aligned bounding-box overlaps arising from proxy conservatism on concave objects; all configurations have zero true mesh-mesh intersections.

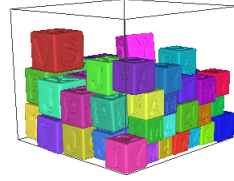
Dataset	n	Container	Pack Eff.%	Hull Dens.%	Cont. Util.%	Overall%	h	AABB Ov.	Mesh Ov.	Time
Block	10	$2.0 \times 1.5 \times 2.0$	79.8	74.4	73.7	43.2	0.87	0	0	37s
	30	$2.5 \times 3.0 \times 2.5$	71.0	66.4	77.6	40.9	0.89	0	0	50s
	60	$3.3 \times 2.8 \times 3.3$	66.7	62.4	90.2	44.6	0.88	2	0	131s
	100	$4.1 \times 2.8 \times 4.1$	74.3	69.6	80.3	44.9	0.90	5	0	184s
Gear	10	$1.5 \times 1.0 \times 1.5$	69.9	35.5	69.0	17.9	0.38	0	0	28s
	30	$2.0 \times 1.5 \times 2.0$	60.2	33.6	98.6	20.4	0.41	16	0	42s
	60	$2.5 \times 1.5 \times 2.5$	55.9	31.7	107.9	24.6	0.41	16	0	129s
	100	$3.5 \times 2.5 \times 3.5$	50.7	29.2	58.1	11.8	0.40	7	0	106s
Kitchen	10	$2.0 \times 1.0 \times 2.0$	52.7	49.0	94.2	26.7	0.58	0	0	20s
	30	$2.8 \times 1.4 \times 2.8$	51.2	47.0	89.9	29.1	0.55	0	0	43s
	60	$3.0 \times 1.7 \times 3.0$	48.8	45.1	107.9	36.4	0.55	4	0	107s
	100	$3.0 \times 2.7 \times 3.0$	49.9	46.1	104.8	36.4	0.56	26	0	215s



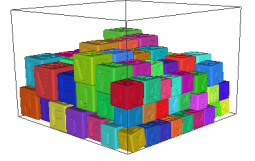
(a) Block $n=10$



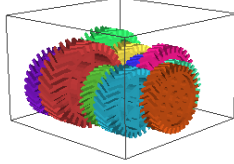
(b) Block $n=30$



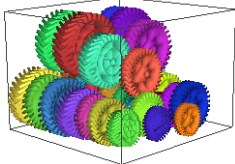
(c) Block $n=60$



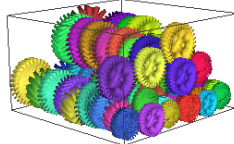
(d) Block $n=100$



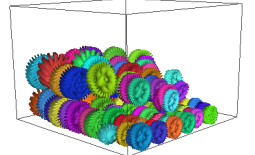
(e) Gear $n=10$



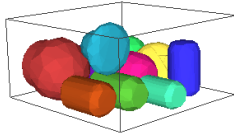
(f) Gear $n=30$



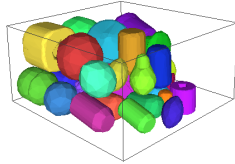
(g) Gear $n=60$



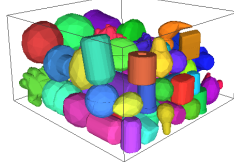
(h) Gear $n=100$



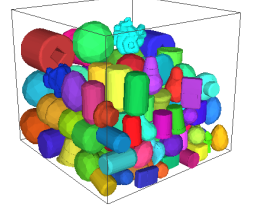
(i) Kitchen $n=10$



(j) Kitchen $n=30$



(k) Kitchen $n=60$



(l) Kitchen $n=100$

Figure 6: Fixed-container (no-squeeze) packings. Mixed-scale objects with random scale in $[0.4, 0.8]$. All configurations have zero mesh-mesh overlap despite AABB overlaps reported in Table 2. Note the emergent multi-layer stacking, particularly visible in blocks at small n .

7. Comparison with Time-Matched Baselines

We compare against three standard baselines on three datasets at four object counts. Protocol: identical inputs (same meshes, same five seeds), wall-clock budget for metaheuristic baselines matched to the median runtime of our method.

Baselines. **DBLF** [3] places objects sequentially at the deepest, bottom-most, left-most extreme point admitting zero overlap, implemented with the full extreme-point set of Crainic et al. [11] and 24 canonical orientations per object. **SA** runs simulated annealing on both translations and rotations from random initialisation, with adaptive step size and an overlap-volume penalty. **BLF+SA** initialises from DBLF and refines with SA. We use the same vertex transformation and AABB evaluation as our method, so all four reported volumes are computed identically.

Table 3: Final container volume (lower is better). Mean and standard deviation over 5 seeds. Bold: best per row.

Dataset	N	DBLF	SA	BLF+SA	Ours
Block	10	4.59 $\pm$ 1.05	7.60 $\pm$ 2.46	4.59 $\pm$ 1.05	5.19 $\pm$ 2.06
	30	16.39 $\pm$ 1.39	26.78 $\pm$ 4.61	16.39 $\pm$ 1.39	16.18 $\pm$ 3.32
	60	35.08 $\pm$ 1.88	53.49 $\pm$ 5.45	35.08 $\pm$ 1.88	33.49 $\pm$ 0.99
	100	60.34 $\pm$ 2.20	90.68 $\pm$ 4.78	60.34 $\pm$ 2.20	53.96 $\pm$ 2.36
Gear	10	2.24 $\pm$ 0.59	2.15 $\pm$ 0.48	1.80 $\pm$ 0.35	1.63 $\pm$ 0.48
	30	8.20 $\pm$ 1.09	10.35 $\pm$ 1.80	8.20 $\pm$ 1.09	8.01 $\pm$ 1.18
	60	18.44 $\pm$ 0.76	25.35 $\pm$ 3.70	18.44 $\pm$ 0.75	16.07 $\pm$ 1.35
	100	29.60 $\pm$ 2.54	42.86 $\pm$ 4.15	29.60 $\pm$ 2.54	24.12 $\pm$ 1.54
Kitchen	10	3.47 $\pm$ 0.89	2.35 $\pm$ 0.55	2.44 $\pm$ 0.67	3.58 $\pm$ 1.23
	30	13.44 $\pm$ 1.75	12.33 $\pm$ 1.50	12.98 $\pm$ 1.60	12.03 $\pm$ 2.36
	60	28.37 $\pm$ 1.48	36.36 $\pm$ 3.60	28.37 $\pm$ 1.48	22.16 $\pm$ 2.69
	100	49.38 $\pm$ 2.77	66.63 $\pm$ 6.19	49.38 $\pm$ 2.77	36.52 $\pm$ 1.46

Result and observations. Our method produces the smallest container at every $(N, \text{dataset})$ pair for $N \geq 30$, with reductions of 11–22% over DBLF and 32–45% over time-matched SA at $N=100$. At $N=10$, the small search space favors the metaheuristic baselines on two of three datasets. Three observations: (1) DBLF and BLF+SA produce identical numbers at $N \geq 30$, meaning the SA refinement layer found zero improving moves on top of DBLF’s construction within the time budget; (2) time-matched SA from random initialisation degrades with N on block and gear, returning containers *larger* than the initial estimate at $N \geq 60$; (3) our advantage grows with N , from a tie at $N=30$ on block to 13% reduction at $N=100$, and from 1% at $N=30$ on kitchen to 26% at $N=100$.

Baseline asymmetries. The baselines use a more constrained rotation search than our method’s continuous gradient optimization; some of our advantage at large N is therefore attributable to the additional effective degrees of freedom rather than to the squeeze mechanism alone.

Note on overlap. Constructive baselines produce zero AABB overlap by construction. Our method’s residual AABB overlap is non-zero (typically 3–5% of pairs), but every result has zero true mesh intersection: the AABBs of two concave objects can touch while the meshes do not. A tighter overlap surrogate is discussed in Section 9.

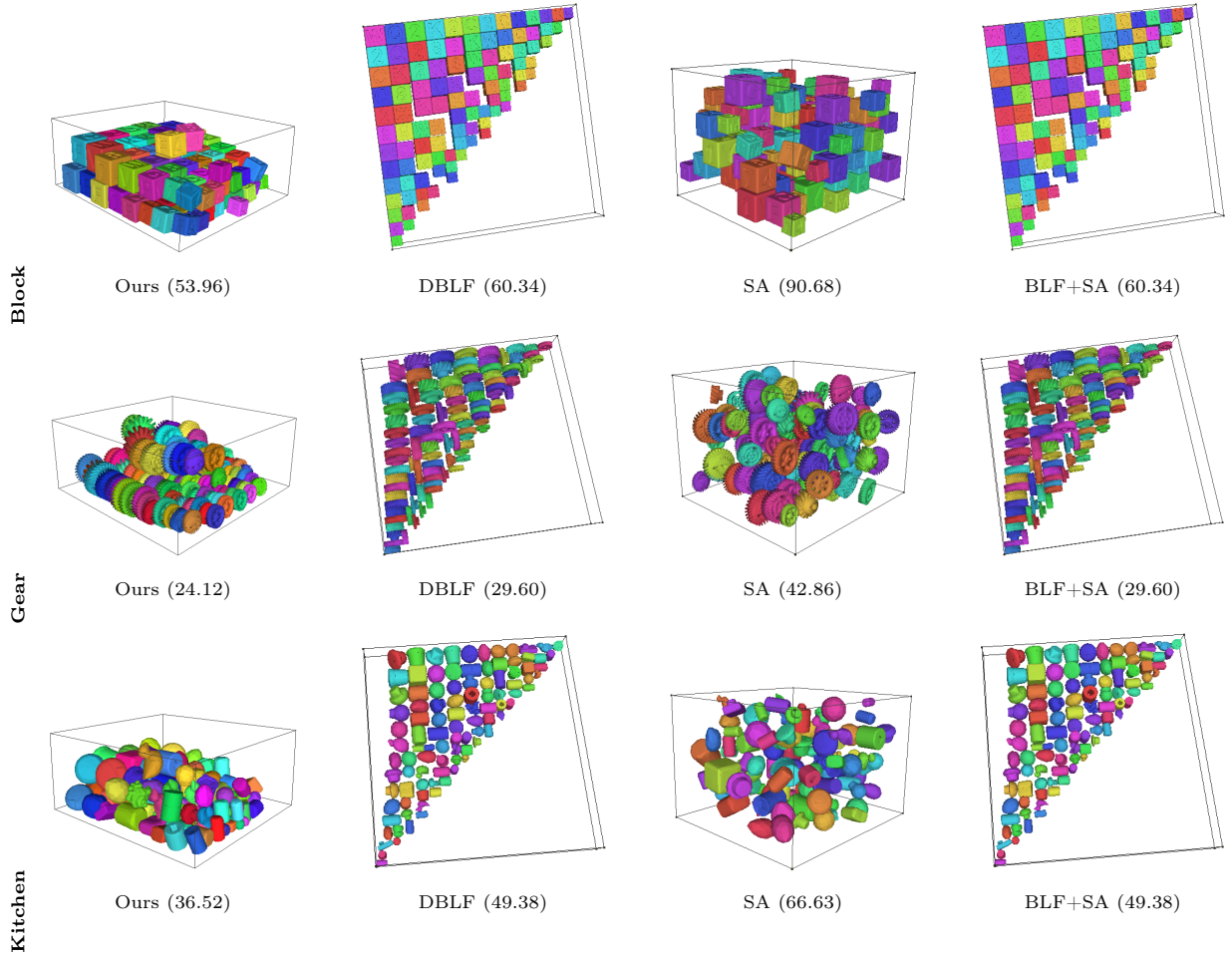


Figure 7: Visual comparison at $N=100$, seed 0. Container volumes in parentheses. Our method (left column) yields the most compact arrangement on every dataset.

Table 4: Mean wall-clock time (s) per run, averaged across block, gear, and kitchen at each N . SA and BLF+SA are budget-matched to Ours.

Method	$N=10$	$N=30$	$N=60$	$N=100$
DBLF	0.3	6.0	44.8	202.3
SA	41.0	81.9	140.9	231.2
BLF+SA	41.2	87.9	185.8	433.8
Ours	40.9	81.3	131.1	239.8

8. Ablation Study

We disable one component at a time to isolate what each contributes. All runs use the kitchen dataset at $N=60$ with five random seeds. Three metrics are reported: container volume (tightness), overlap count (feasibility), and floating count (objects whose bottom sits more than 0.05 units above any supporting surface; physical plausibility). A well-factorized loss decomposition should have the property that each ablation damages a *different* metric.

Table 5: Ablation at $N=60$. Mean $\pm$ std, 5 seeds.

Variant	Vol	Ov.	Float
Full	19.12 $\pm$ 1.67	2.2 $\pm$ 2.5	0.2$\pm$0.4
No squeeze	18.32$\pm$1.29	4.0 $\pm$ 1.7	1.4 $\pm$ 0.5
No gravity	20.74 $\pm$ 2.59	0.6$\pm$0.5	23.8 $\pm$ 1.2
No coh./cent.	19.95 $\pm$ 1.45	1.4 $\pm$ 1.0	1.0 $\pm$ 1.3
No rotation	19.20 $\pm$ 1.94	6.4 $\pm$ 2.2	1.4 $\pm$ 1.0

Ablation Study — $n = 60$, mean $\pm$ std over 5 seeds

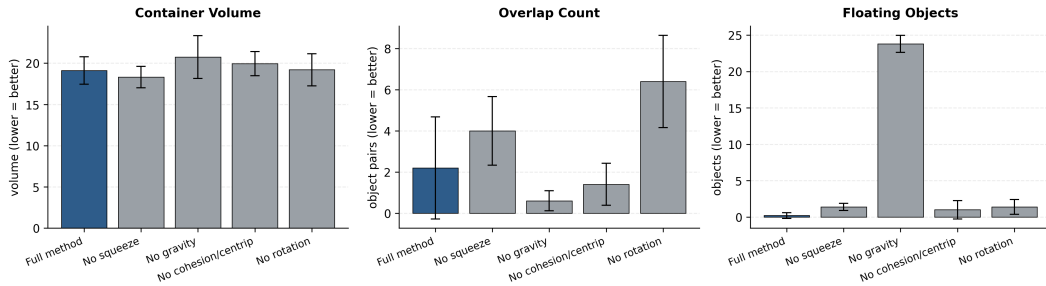


Figure 8: Ablation metrics; full method (blue) vs ablations (grey).

Reading the variants. *No gravity* removes the vertical pull toward supporting surfaces. The floating count rises from 0.2 to 23.8 out of 60 objects, while container volume and overlap remain essentially unchanged.

No rotation constrains the optimizer to three translational degrees of freedom; container volume is barely affected, but overlap count rises from 2.2 to 6.4 pairs, since the optimizer cannot rotate objects into interlocking configurations. *No cohesion and centripetal* removes the soft regularisers; container volume rises from 19.12 to 19.95 (+4%) because the contact term alone cannot pull a fragmented cluster back together.

No squeeze is more subtle. The reported 18.32 is numerically smaller than the full method’s 19.12, but this does not mean squeezing hurts. Without squeeze the container remains at the initial estimate throughout training, and the reported volume is the tight AABB of the converged cluster inside that oversized envelope. The full method pulls the container down during optimization, and the cluster at convergence is denser and vertically taller, yielding a slightly larger tight AABB. The consequences of disabling squeeze are captured in container utilization (Section 5): with squeeze disabled, the cluster either leaves substantial slack or overflows, whereas the full method consistently achieves 81 to 95% utilization.

Summary. Every component addresses a distinct failure: gravity for stability, rotation for feasibility, cohesion/centripetal for volume tightening, and squeeze for container sizing. Each ablation breaks a different metric, confirming a well-factorized design.

9. Limitations

AABB overlap surrogate. Overlap is computed between axis-aligned bounding boxes, not between meshes. For irregular meshes, this is a strict upper bound on true mesh overlap, so the optimizer can converge with small residual AABB overlap while meshes do not intersect. A tighter surrogate based on oriented bounding boxes or a differentiable signed distance field [25] would further reduce container size at the cost of additional per-epoch computation.

Small N . At $N \leq 10$, simulated annealing outperforms our method on some datasets. The gradient-based approach has its advantage at scale ($N \geq 60$), where the high-dimensional configuration space overwhelms random perturbation but the gradient still carries useful information.

Published-method comparison. Direct head-to-head comparison with recent developments was not possible because the specific 3D models used in their experiments are not released as a public dataset. We therefore compare only against baselines we can re-implement on our datasets. Context from their published numbers is available in the respective papers.

Rectangular container. The formulation assumes a right rectangular prism container. Extending the boundary loss to general convex containers defined by a signed distance function is straightforward in principle but was not pursued here.

Off-line setting. The method is an off-line, all-at-once packer. It does not address the online variant, for which learning-based approaches retain a structural advantage.

Future work. Promising extensions: GPU-level parallelization for several hundred objects; differentiable SDF or OBB overlap surrogates; non-rectangular containers; and a light-weight learned initializer that warm-starts the optimizer.

10. Conclusion

We presented a differentiable optimization framework for packing irregular 3D objects that jointly estimates all three container dimensions. The formulation combines six physics-inspired loss terms with an adaptive container squeezing mechanism whose pair-count-scaled threshold makes the same pipeline work reliably from $N=10$ to $N>100$ without per-instance tuning. Vectorizing every pairwise computation yields a 3.4-54-fold speedup over the reference Python loop, and the full pipeline finishes a 100-object instance in under 4 minutes on a single GPU using only Python and PyTorch. Across four object categories, the method produces overlap-free arrangements with containers 11 to 32% smaller than time-matched DBLF and SA baselines at $N \geq 60$. The Python-only design removes the C++, CUDA, and PhysX dependency of the strongest existing pipelines, which we expect to make the method substantially easier to integrate into larger differentiable systems.

Acknowledgements

The first author gratefully acknowledges the support of the India AI Fellowship under the IndiaAI Mission.

References

- [1] A. Bortfeldt, G. Wäscher, Constraints in container loading: A state-of-the-art review, *European Journal of Operational Research* 229 (1) (2013) 1–20.
- [2] X. Zhao, J.A. Bennell, T. Bektaş, K. Dowsland, A comparative review of 3D container loading algorithms, *International Transactions in Operational Research* 23 (1–2) (2016) 287–320.
- [3] L. Wang, S. Guo, S. Chen, W. Zhu, A. Lim, Two natural heuristics for 3D packing with practical loading constraints, in: *PRICAI*, 2010, pp. 256–267.
- [4] F. Wang, K. Hauser, Dense robotic packing of irregular and novel 3D objects, *IEEE Transactions on Robotics* 38 (2) (2022) 1160–1173.

- [5] K. Tole, F. Heurtefeux, Y. Chen, A first-time fit grid search algorithm with simulated annealing for the 3D packing problem, *Applied Soft Computing* (2023).
- [6] Q. Zhuang, Z. Chen, K. He, J. Cao, W. Wang, Dynamics simulation-based packing of irregular 3D objects, *Computers & Graphics* 123 (2024) 103996.
- [7] Y. Ma, Z. Chen, W. Hu, W. Wang, Packing irregular objects in 3D space via hybrid optimization, *Computer Graphics Forum* 37 (5) (2018) 49–59.
- [8] T. Romanova, J. Bennell, Y. Stoyan, A. Pankratov, Packing of concave polyhedra with continuous rotations using nonlinear optimisation, *European Journal of Operational Research* 268 (1) (2018) 37–53.
- [9] Q. Cui, V. Rong, D. Chen, W. Matusik, Dense, interlocking-free and scalable spectral packing of generic 3D objects, *ACM Transactions on Graphics* 42 (4) (2023) 1–14.
- [10] H. Zhao, Z. Pan, Y. Yu, K. Xu, Learning physically realizable skills for online packing of general 3D shapes, *ACM Transactions on Graphics* 42 (5) (2023) 165:1–21.
- [11] T.G. Crainic, G. Perboli, R. Tadei, Extreme point-based heuristics for three-dimensional bin packing, *INFORMS Journal on Computing* 20 (3) (2008) 368–384.
- [12] H. Gehring, A. Bortfeldt, A parallel genetic algorithm for solving the container loading problem, *International Transactions in Operational Research* 9 (4) (2002) 497–511.
- [13] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: *ICLR*, 2015.
- [14] H. Zhao, Q. She, C. Zhu, Y. Yang, K. Xu, Online 3D bin packing with constrained deep reinforcement learning, in: *Proceedings of the AAAI Conference on Artificial Intelligence* 35 (1) (2021) 741–749.
- [15] Q. Hu, A. Zhu, J. Yin, Solving a new 3D bin packing problem with deep reinforcement learning method, *arXiv preprint arXiv:1708.05930* (2017).
- [16] L.J. Araújo, E. Özcan, J.A. Atkin, M. Baumers, Analysis of irregular three-dimensional packing problems in additive manufacturing: a new taxonomy and dataset, *International Journal of Production Research* 57 (18) (2019) 5920–5934.
- [17] M. Dawson-Haggerty et al., trimesh: Load and analyze triangular meshes, version 3.2.0, 2019. <https://trimesh.org/>
- [18] Y. Stoyan, A. Pankratov, T. Romanova, Cutting and packing problems for irregular objects with continuous rotations: Mathematical modelling and non-linear optimization, *Journal of the Operational Research Society* 67 (5) (2016) 786–800.

- [19] M. Calabrese, T. Primo, A. Del Prete, Nesting algorithm for optimization part placement in additive manufacturing, *International Journal of Advanced Manufacturing Technology* 119 (2022) 4613–4634.
- [20] E. Hopper, B.C.H. Turton, A review of the application of meta-heuristic algorithms to 2D strip packing problems, *Artificial Intelligence Review* 16 (4) (2001) 257–300.
- [21] M.R. Garey, D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, New York, 1979.
- [22] F. Wang, K. Hauser, Robot packing with known items and nondeterministic arrival order, in: *Robotics: Science and Systems*, 2019.
- [23] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, PyTorch: An imperative style, high-performance deep learning library, in: *Advances in Neural Information Processing Systems* 32 (2019).
- [24] A.J.S. Leao, F. Toledo, J.F. Oliveira, M.A. Carravilla, R. Alvarez-Valdes, Irregular packing problems: A review of mathematical models, *European Journal of Operational Research* 282 (3) (2020) 803–822.
- [25] J.J. Park, P. Florence, J. Straub, R. Newcombe, S. Lovegrove, DeepSDF: Learning continuous signed distance functions for shape representation, in: *CVPR*, 2019, pp. 165–174.
- [26] M. Attene, Shapes in a box: Disassembling 3D objects for efficient packing and fabrication, *Computer Graphics Forum* 34 (8) (2015) 64–76.
- [27] V. Krs, R. Měch, M. Gaillard, N. Carr, B. Benes, PICO: Procedural iterative constrained optimizer for geometric modeling, *IEEE Transactions on Visualization and Computer Graphics* 27 (10) (2021) 3968–3981.
- [28] L. Cao, L. Tian, H. Peng, Y. Zhou, L. Lu, Constrained stacking in DLP 3D printing, *Computers & Graphics* 95 (2021) 60–68.